

ForgeCLI – Technical Design Specification

ForgeCLI – Technical Design Specification for AI-Assisted Development

Version: 1.0

Target Audience: AI Coding Agents (Claude Code, Codex, Cursor, Continue, OpenHands)

Project Type: Open Source CLI Application

Primary Language: Go

Architecture Style: Modular / Plugin-Oriented / Unix-Native

1. Project Overview

ForgeCLI is a lightweight, modular, terminal-based data transformation and analysis framework inspired by CyberChef.

The project focuses on:

- automatic encoding and format detection
- recursive analysis
- forensic-friendly workflows
- streaming support
- high performance
- low memory usage
- scriptability
- extensibility via plugins
- fully offline operation

2. Core Design Principles

2.1 Lightweight First

- fast startup time
- minimal dependencies
- low memory allocations
- efficient streaming
- small binary size
- minimal runtime overhead

Target metrics:

- Startup Time: <100ms
- Idle RAM Usage: <30MB

- Binary Size: <20MB

2.2 Unix Philosophy

Everything must work correctly with:

- stdin
- stdout
- stderr
- pipes
- redirection
- shell scripting

Examples:

```
cat sample.txt | forge detect
```

```
curl https://example.com | forge auto
```

```
forge decode base64 payload.txt
```

2.3 AI-Friendly Architecture

Repository and code organization must support:

- AI-assisted coding
- modular reasoning
- isolated feature generation
- incremental implementation

3. High-Level Architecture

CLI Layer

- > Command Dispatcher
- > Analysis Engine
- > Detection Engine
- > Transformation Engine
- > Recursive Pipeline Resolver
- > Output Formatter
- > Plugin Runtime

4. Technology Stack

Primary Language: Go 1.24+

Recommended Libraries:

- Cobra

- Viper
- zerolog
- testify
- goreleaser
- BubbleTea (optional)
- Lipgloss (optional)

5. Repository Structure

forgecli/

■■■ cmd/

■■■ internal/

■■■ pkg/

■■■ plugins/

■■■ recipes/

■■■ examples/

■■■ tests/

■■■ benchmarks/

■■■ docs/

■■■ .github/

■■■ main.go

■■■ go.mod

■■■ README.md

■■■ CONTRIBUTING.md

■■■ LICENSE

■■■ Makefile

6. Core Functional Requirements

Detection Engine:

- Base64 detection
- Hex detection
- URL encoding
- gzip
- zlib
- JWT
- JSON
- XOR single-byte detection
- UTF encoding detection

- ELF/PE recognition

Recursive Analysis:

- recursive decoding
- nested transformation detection
- recursion depth limits
- transformation trees

Streaming Support:

- chunk processing
- buffered readers
- progressive entropy analysis

7. CLI Specification

Required commands:

- forge detect
- forge auto
- forge decode
- forge encode
- forge inspect
- forge entropy
- forge hash
- forge recipe
- forge plugins

8. Plugin System

Goals:

- third-party detectors
- third-party analyzers
- third-party decoders
- custom recipes

Plugin Interface Example:

```
type Plugin interface {  
    Name() string  
    Version() string  
    Register(reg Registry)  
}
```

9. Performance Requirements

- parallel detection
- worker pools
- bounded concurrency
- reusable buffers
- streaming APIs

10. Security Requirements

ForgeCLI MUST NOT:

- execute payloads
- execute macros
- run shellcode
- auto-open files
- auto-execute binaries

11. Testing Requirements

Required:

- unit tests
- integration tests
- benchmarks
- fuzzing (recommended)

Coverage goals:

- 70%+ global
- 90%+ for critical systems

12. Documentation Requirements

Required docs:

- README.md
- CONTRIBUTING.md
- ARCHITECTURE.md
- PLUGINS.md
- RECIPES.md
- SECURITY.md

13. Coding Standards

Rules:

- descriptive names
- small functions
- avoid giant files
- avoid hidden side effects
- avoid global state

14. Suggested Development Roadmap

Phase 1:

- CLI framework
- Base64 detection
- Hex detection
- gzip support
- entropy analysis

Phase 2:

- streaming analysis
- plugin system
- recipe engine
- XOR bruteforce

Phase 3:

- PE/ELF parsing
- YARA integration
- TUI mode

Phase 4:

- WASM plugins
- AI-assisted suggestions
- distributed analysis

15. Long-Term Vision

ForgeCLI should become:

- a terminal-native CyberChef alternative

- a standard quick-analysis tool
- a modular analysis platform
- a community-driven ecosystem
- a high-performance forensic utility